



BUSINESS REQUIREMENTS DOCUMENT

Synthetic Monitoring Platform

Core objectives, scope, and functional requirements.

PREPARED FOR Client Stakeholders • PREPARED BY Agam AI Labs • August 24, 2026

REF. AGAL-2026-ALLOCABS-SM-001.01 – CLIENT DRAFT



Contents

1	Executive Summary	2
1.1	Purpose of the Document	2
1.2	Business Problem Statement	2
1.3	Proposed Solution	2
2	Project Overview & Objectives	3
2.1	Business Objectives	3
2.2	Success Metrics (KPIs)	4
3	Scope	4
3.1	In-Scope	4
3.2	Out-of-Scope	5
4	User Personas & Roles	6
4.1	System Administrators (The “Configurators”)	6
4.2	Application Owners (The “Responders”)	6
4.3	Executive Stakeholders (The “Reviewers”)	7
5	Functional Requirements (The Core Engine)	7
5.1	Probe & Check Execution Engine	7
5.2	Alerting & Routing Matrix	8
5.3	Unified Service Dashboard	8
5.4	User & Role Management (RBAC)	9
6	Non-Functional Requirements	9
6.1	Performance & Scalability	9
6.2	Security & Compliance	9
6.3	Availability & Reliability	10
7	Dependencies & Assumptions	10
7.1	Third-Party Integrations	10
7.2	Technical Assumptions	11

1 | Executive Summary

1.1 Purpose of the Document

This Business Requirements Document (BRD) defines the objectives, scope, and functional requirements for a centralized Synthetic Monitoring Platform. It aligns engineering and client stakeholders on what the system will do, who will use it, and how it fits existing workflows before development begins.

1.2 Business Problem Statement

The client's applications run across a mix of hosting providers and infrastructure types, creating a critical visibility gap:

ECOSYSTEM CHALLENGES

Critical Visibility Gaps

- **Siloed Monitoring:** Each hosting provider only sees its own slice, never the health of the ecosystem as a whole.
- **Delayed Detection:** With no outside-in checks, end-users are often the first to report downtime.
- **Fragmented Alerting:** No standard way to identify the failing app, find its owner, and alert them fast.

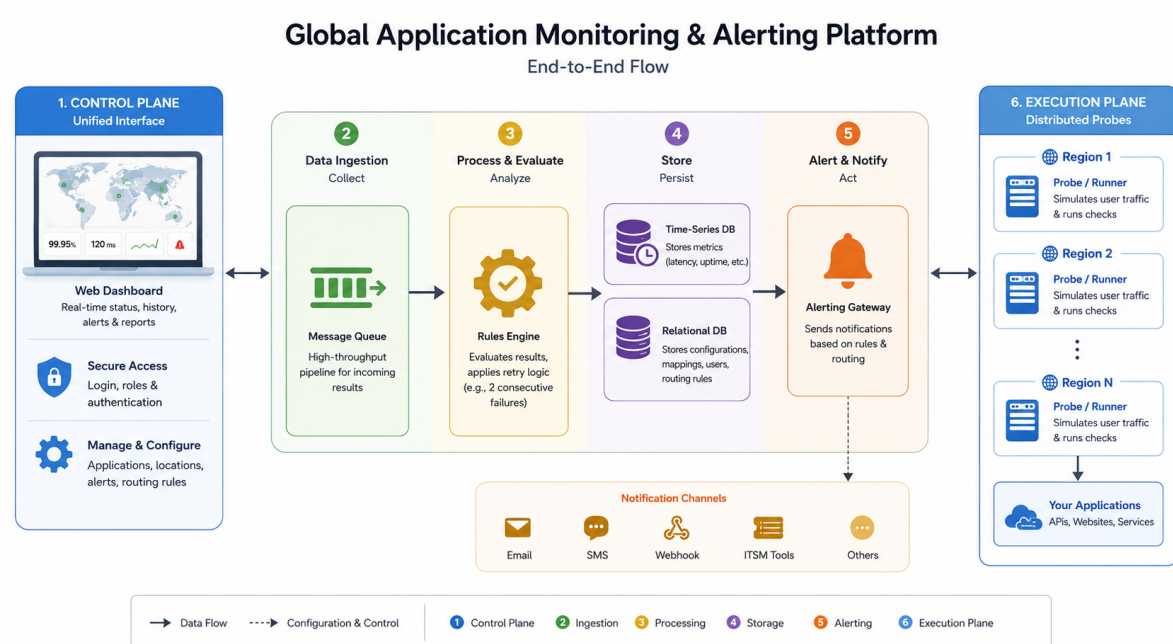
1.3 Proposed Solution

A unified synthetic monitoring platform that acts as the single source of truth for application health, regardless of where an application is hosted.

STRATEGIC APPROACH

Unified Architecture

- **Active Simulation:** Distributed probes continuously simulate real user traffic – pings, HTTP requests, API calls – against every application.
- **Centralized Visibility:** Uptime and latency data from every environment rolls up into one dashboard.
- **Intelligent Routing:** On failure, a rules engine validates the issue and alerts the right owner by SMS, email, or webhook, cutting Mean Time to Detect (MTTD).



2 | Project Overview & Objectives

2.1 Business Objectives

This platform moves the client from reactive, decentralized monitoring to a proactive, centralized model:

CORE OBJECTIVES

Strategic Platform Goals

- **One View, Every Environment:** A single, standardized picture of application health across all hosting providers.
- **Standardized Response:** A routing matrix that automatically finds and notifies the correct owner on failure.
- **Get Ahead of Issues:** Catch degradation from the end-user's perspective, before customers report it.

2.2 Success Metrics (KPIs)

PERFORMANCE TARGETS

Key Performance Indicators

- **Mean Time to Detect (MTTD):** Under 5 minutes for tier-1 applications.
- **Proactive Detection Rate:** Over 95% of outages caught before end-users report them.
- **Alert Delivery Latency:** SMS/Email delivered within 60 seconds of a validated failure.
- **Platform Availability:** 99.99% uptime for the monitoring platform itself.

3 | Scope

3.1 In-Scope

The initial release includes:

INCLUDED CAPABILITIES

In-Scope Features

- **Outside-In Monitoring:** HTTP/HTTPS, TCP port checks, DNS resolution, and multi-step REST API sequences.
- **Alert Routing Engine:** Configurable retry logic (e.g., fail twice before alerting) plus tag-based mapping of apps to owners.
- **Notification Channels:** SMS, Email, and generic Webhooks for standard ITSM tools.
- **Unified Dashboard:** A web-based UI for real-time status, historical latency, and configuration.
- **Role-Based Access Control (RBAC):** Basic segregation between administrators and read-only stakeholders.

3.2 Out-of-Scope

To keep this phase focused, the following are explicitly excluded:

EXPLICIT EXCLUSIONS

Out-of-Scope Items

- **Application Performance Monitoring (APM):** Code profiling, memory leaks, or query analysis.
- **Infrastructure Monitoring:** Server-level CPU, RAM, or disk metrics.
- **Log Aggregation:** Ingesting or searching application logs (e.g., Splunk/ELK).
- **Auto-Remediation:** Automated recovery actions, such as restarting a container or server.

4 | User Personas & Roles

Three roles interact with the platform, each with distinct needs and permissions.

4.1 System Administrators (The “Configurators”)

PERSONA 01

System Administrator Profile

Who they are: IT Operations, SRE, or NOC engineers who run the platform itself – onboarding applications, setting routing rules, managing integration credentials, and administering access.

What they need:

- Bulk onboarding, and deep diagnostics to pinpoint why a probe is failing (e.g., SSL vs. DNS errors).
- Full CRUD access across every module, with an unrestricted view of the global matrix.

4.2 Application Owners (The “Responders”)

PERSONA 02

Application Owner Profile

Who they are: Developers, product managers, or business-unit leads who own an application and triage its incidents.

What they need:

- Accurate alerts with minimal false positives – alert fatigue is their biggest risk.
- Actionable detail in the alert itself (e.g., “HTTP 502 at 10:04 AM, AWS us-east-1”).
- Read-only access scoped to just their own applications.

4.3 Executive Stakeholders (The “Reviewers”)

PERSONA 03

Executive Stakeholder Profile

Who they are: CIOs, CTOs, or IT Directors tracking the health of technology investments and vendor performance across the business.

What they need:

- Simple red/yellow/green status boards by category (e.g., all e-commerce apps).
- Automated weekly/monthly uptime reports – with no configuration access, to avoid accidental changes.

5 | Functional Requirements (The Core Engine)

5.1 Probe & Check Execution Engine

Independently simulates and validates user traffic across every hosting environment.

EXECUTION ENGINE

Simulation & Validation

- **Protocol Support:** HTTP/HTTPS (GET/POST/PUT/DELETE), TCP port checks, DNS resolution, and ICMP ping.
- **Multi-Step API Sequences:** Chain calls and pass tokens between them (e.g., OAuth login) to validate full workflows.
- **Execution Parameters:** Per-app check intervals – 1-minute, 5-minute, or custom cron schedules.
- **Geographic Distribution:** Probes deployable across multiple regions to validate availability and measure latency.

5.2 Alerting & Routing Matrix

Routes validated failures directly to the person who can act on them.

INCIDENT ROUTING

Alerts & Notifications

- **Validation & Deduplication:** Configurable retry thresholds (e.g., 2 consecutive failures from 2 locations) before an alert fires.
- **Tag-Based Routing:** Metadata tags (Owner, Priority) map each app's failures to the right destination.
- **Notification Channels:** SMTP, SMS gateways (e.g., Twilio), and webhooks into ServiceNow, Jira, or Slack.
- **Escalation Policies:** Unacknowledged alerts auto-escalate to a secondary contact after a configurable window (e.g., 15 minutes).

5.3 Unified Service Dashboard

One interface for status, diagnostics, and reporting.

USER INTERFACE

Single Pane of Glass

- **Global Status Board:** Real-time, color-coded health for every application, filterable by host, environment, or tag.
- **Forensic Incident Logs:** Exact error payload, response code, latency, and SSL status for any failed check.
- **SLA & Reporting:** Uptime and latency averages over 7/30/90-day periods, exportable as PDF/CSV.
- **Configuration Interface:** Create, duplicate, modify, or pause checks without writing code.

5.4 User & Role Management (RBAC)

ACCESS CONTROL

Security & Roles

- **Role Segregation:** Three tiers – System Administrator (full CRUD), Application Owner (read-only + acknowledge), Executive Stakeholder (read-only).
- **Single Sign-On (SSO):** Integrates with the client's identity provider (e.g., Okta, Azure AD) via SAML 2.0 or OIDC.

6 | Non-Functional Requirements

6.1 Performance & Scalability

SYSTEM CONSTRAINTS

Performance Targets

- **High Throughput:** Scales to at least 10,000 synthetic checks per minute.
- **Low Latency Processing:** Under 500ms from a probe detecting a failure to the alert being queued.

6.2 Security & Compliance

COMPLIANCE

Security Standards

- **Data Encryption:** AES-256 at rest; TLS 1.2 or higher in transit.
- **Secrets Management:** Probe credentials (API tokens, passwords) live in an encrypted vault, injected only at runtime.
- **Network Isolation:** Internal probes use strict outbound-only connections – no inbound access into the client’s network.

6.3 Availability & Reliability

RELIABILITY

Uptime & Redundancy

- **High Availability:** Active-active deployment across multiple availability zones.
- **Uptime Guarantee:** 99.99% SLA; checks auto-redistribute if a probe runner goes offline.

7 | Dependencies & Assumptions

7.1 Third-Party Integrations

Alerting depends on the following external services being in place:

EXTERNAL DEPENDENCIES

Integration Requirements

- **Notification Gateways:** Active accounts/API keys for SMS (e.g., Twilio, AWS SNS) and an SMTP relay for email.
- **ITSM Integration:** The client’s ticketing system (e.g., ServiceNow, PagerDuty,

Jira) must accept standard JSON payloads via a secure endpoint.

- **Identity Management:** A SAML 2.0 or OIDC-compliant identity provider for SSO.

7.2 Technical Assumptions

SYSTEM ASSUMPTIONS

Technical Prerequisites

- **Endpoint Testability:** Applications expose reachable endpoints (e.g., /health, standard routes) that can be queried safely, without side effects.
- **Firewall Egress:** The client's network team will grant outbound exceptions for probes inside private network boundaries.
- **Hosting Agnosticism:** No agents are installed on client host machines – the architecture stays strictly outside-in.